

Introduction to Algorithm

기초 알고리즘

Intractable 문제 해결 접근

근사 알고리즘 (Approximation Algorithm)

NP-Complete 문제의 벽

- SAT, TSP, 0/1 Knapsack 등 NP-Complete 문제들은 입력 크기가 커짐에 따라 실행 시간이 지수적으로 증가하여 슈퍼컴퓨터로도 해결이 불가능
- 현실의 비즈니스 문제는 매분 매초 결정이 필요한데, 이론적 최적해만 기다릴 수는 없음
- 따라서, '시간'을 얻기 위해 '정확도'를 일부 양보하는 전략이 필요

현실세계의 제약 조건: 시간과 자원

- 실제 산업 현장에서는 데이터가 수만 개에서 수백만 개에 달하는 경우가 많음
 - 예) 전국 물류 배송 경로 최적화: 수천 개의 지점의 방문순서 최적화
- 따라서, 입력의 크기에 따라 실행시간이 급속도로 증가하는 지수 시간 알고리즘은 사실상 사용하기 어려움
- 100%의 정확도를 위해 100년이 걸리는 시스템보다, 98%의 정확도를 위해 0.1초가 걸리는 시스템이 필요

➔ “근사 알고리즘”의 탄생

근사 알고리즘 (Approximation Algorithm)

- 근사 알고리즘은 최적화 문제에 대해 **다항 시간(Polynomial Time) 내에 최적해에 근접한 해**를 찾는 알고리즘
- 단순히 '운 좋게 가까운 답'을 내놓는 것이 아니라, **어떤 입력에 대해서도 일정한 품질을 보장**

➔ 수학적 근거를 바탕으로 최악의 경우에도 정답과 얼마나 차이가 나는지 증명할 수 있는 알고리즘만을 근사 알고리즘이라 할 수 있다

근사 비율 (Approximation Ratio)

- 근사 알고리즘의 성능을 객관적으로 평가하기 위해 사용하며, 알고리즘이 찾아낸 해가 최적해와 얼마나 차이가 나는지를 나타내는 척도
- 근사 비율($p(n)$)은 “최적해의 값”과 알고리즘이 구한 “근사해의 값” 사이의 비율을 의미
- 비율이 1에 가까울수록 알고리즘의 성능이 우수(1인 경우, 최적해와 같음)

근사 비율 (Approximation Ratio)

- 근사 비율($\rho(n)$) = "최적해의 값(C^*)" 과 "근사해의 값(C)" 의 비율

$$\max \left(\frac{C}{C^*}, \frac{C^*}{C} \right) \leq \rho(n)$$

근사 비율 (Approximation Ratio)

- 최소화 문제 (Minimization): $C > C^*$ 또는 $C = C^*$
- 최대화 문제 (Maximization): $C < C^*$ 또는 $C = C^*$

➔ 어느 경우 든, $\rho(n)$ 은 항상 1보다 크거나 같다!

$$\max \left(\frac{C}{C^*}, \frac{C^*}{C} \right) \leq \rho(n)$$

근사 비율 (Approximation Ratio) – 최악의 성능 보장

- $p(n) = 2$ 이라면,
 - 최소화 문제 (Minimization): 근사해는 최적해의 2배 이하
 - 최대화 문제 (Maximization): 근사해는 최적해의 0.5배(절반) 이상
- **$P(n) = 1$ 이라면, 근사해는 최적화와 동일**

$$\max \left(\frac{C}{C^*}, \frac{C^*}{C} \right) \leq \rho(n)$$

대표적인 근사 사례: TSP (Traveling Salesperson Problem)

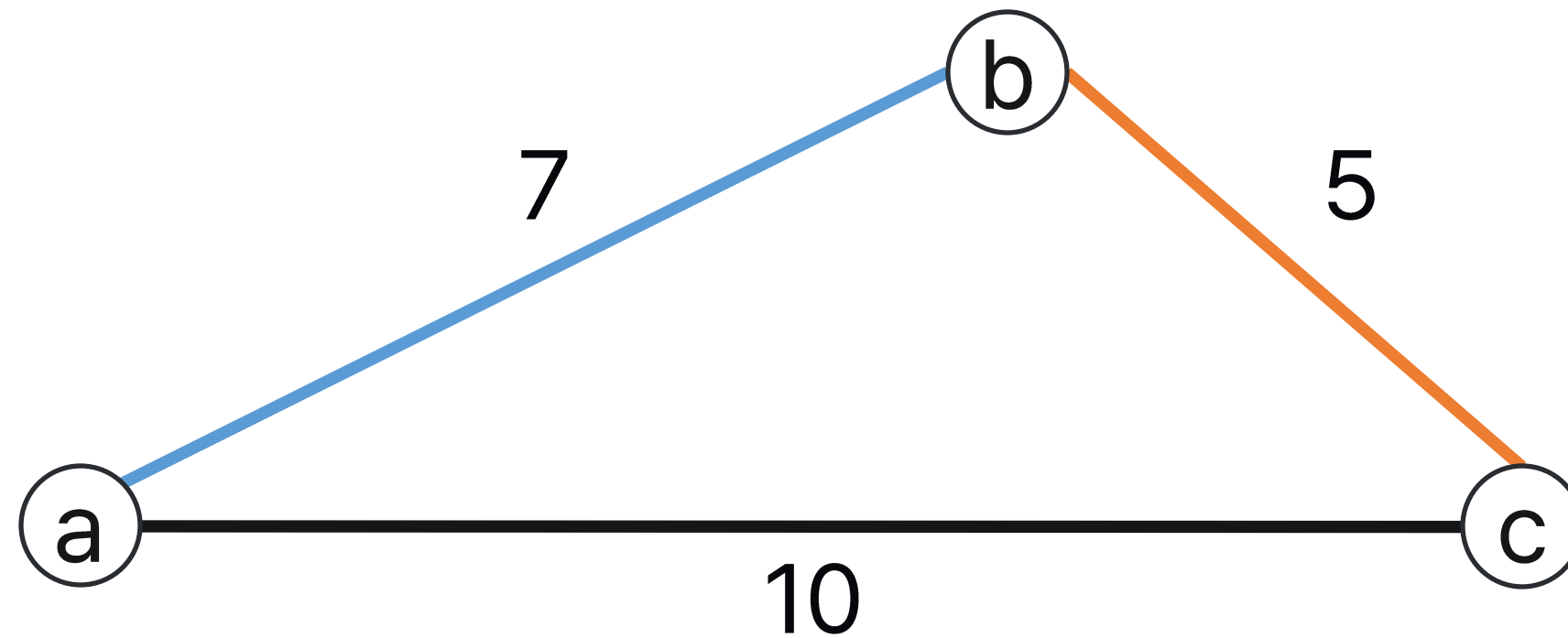
TSP 문제 접근

- 모든 도시를 한 번씩 방문하고 돌아오는 TSP문제는 도시가 20개만 넘어도 풀이시간이 급속히 증가 ($O(n!)$)하여 문제해결이 어려워짐
 - 하지만, (1) 전제 조건을 만족한다면, (2) 최소 신장 트리(MST)를 이용하여 근사해를 찾는 것이 가능
- ➔ 전제 조건: 도시 간의 거리는 삼각 부등식(Triangle Inequality)을 만족한다

TSP 문제 접근

(1) 전제 조건: 도시 간의 거리는 삼각 부등식(Triangle Inequality)을 만족한다

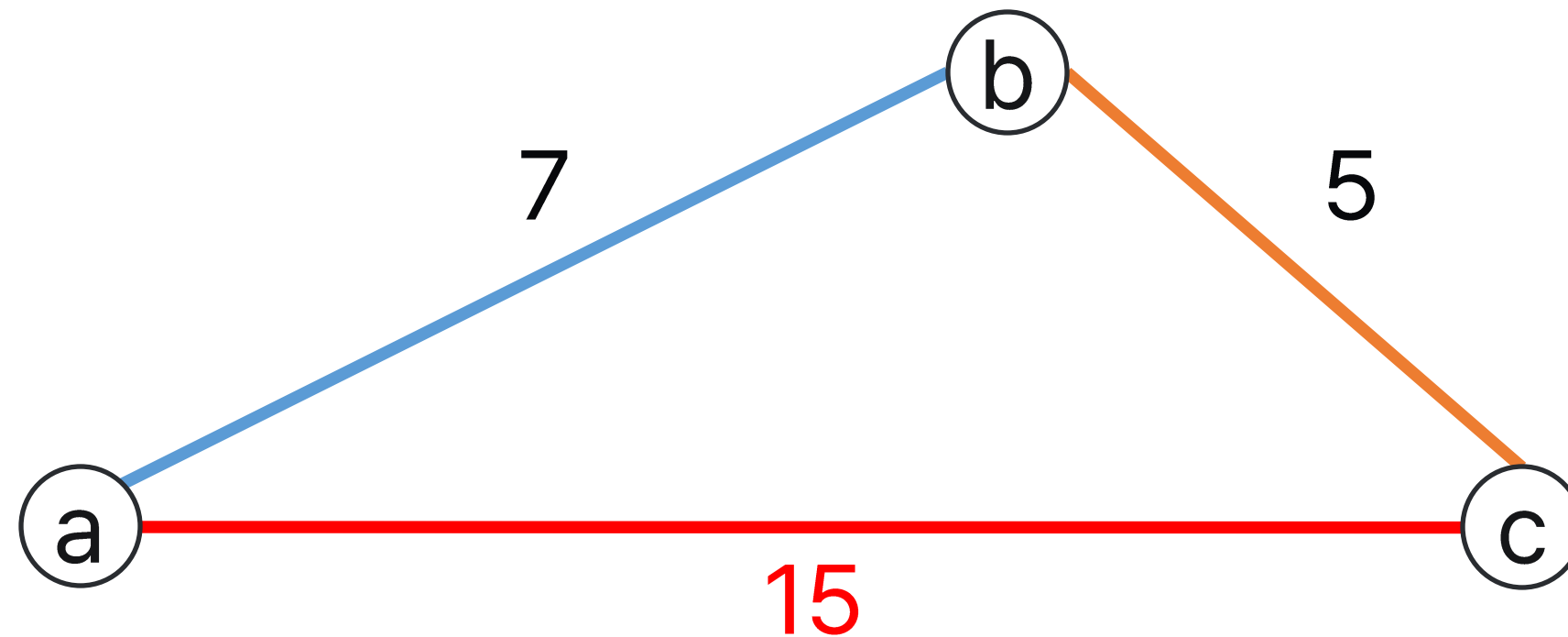
$$\text{dist}(a, c) \leq \text{dist}(a, b) + \text{dist}(b, c)$$



TSP 문제 접근

~~(1) 전제 조건: 도시 간의 거리는 삼각 부등식(Triangle Inequality)을 만족한다~~

$$\text{dist}(a, c) \leq \text{dist}(a, b) + \text{dist}(b, c)$$



TSP 문제 접근

(2) 최소 신장 트리(MST)를 이용하여 근사해를 찾을 수 있다

- TSP 경로는 모든 정점을 연결하는 "사이클"
- 사이클에서 간선 하나를 제거하면, 모든 정점을 연결하는 "트리"
- 모든 정점을 연결하는 가장 비용이 적은 트리 → 최소 신장 트리(MST)
- MST문제는 Kruskal이나 Prim 알고리즘으로 답을 빠르게 구할 수 있음 ($O(E \log V)$)

*MST: 모든 정점을 포함 + 사이클 없음 + 간선들의 가중치 합 최소인 트리

TSP 문제 풀이: MST 기반 근사 알고리즘

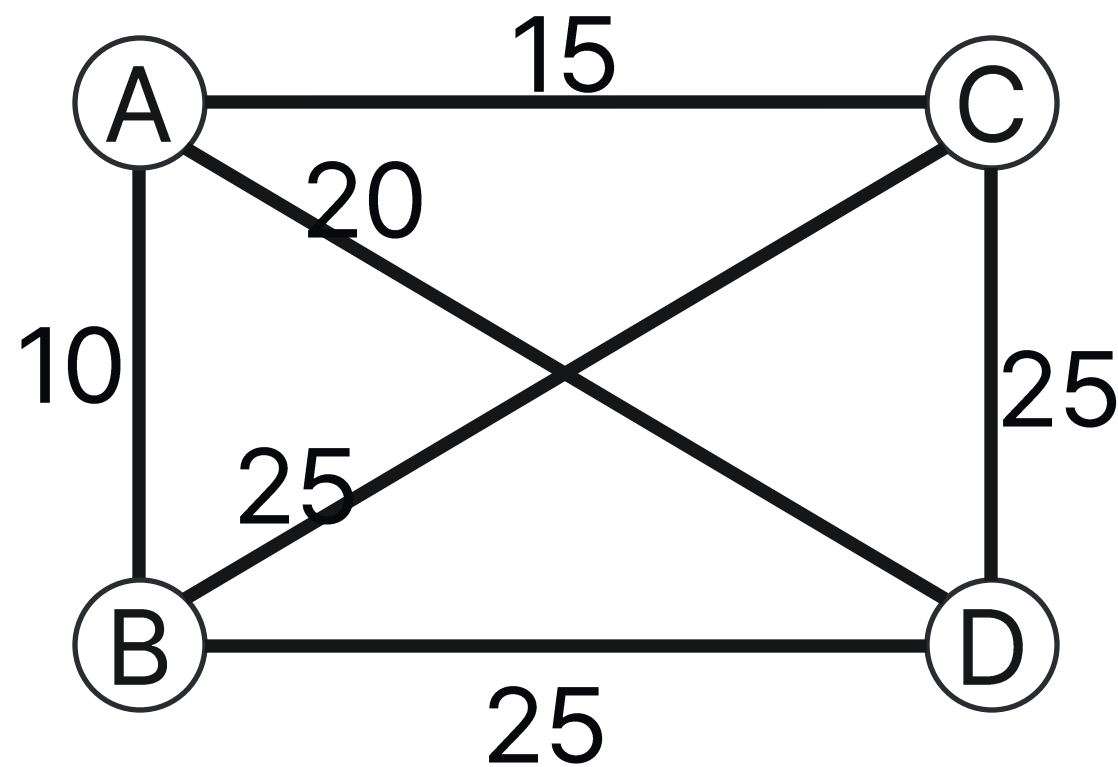
1. 주어진 그래프에서 MST 구축
2. MST 간선을 '왕복' 한다 가정하고, 모든 노드를 방문하는 경로 생성
 - 모든 노드를 방문하고 돌아오지만, 중복 방문하는 도시가 많아 비효율적인 상태
3. 지름길(Shortcut) 탐색: 중복 방문 도시는 건너뛰고, 방문하지 않은 다음 도시로 이동
 - 예) $a \rightarrow b \rightarrow c$ 인 경우, $a \rightarrow c$ 로 직접 이동
4. 지름길을 적용하면, 중복 없는 TSP경로 완성

TSP 문제 예시

- 문제: 아래와 같이 도시 A, B, C, D 와 도시간 거리가 주어진다. 모든 도시를 한 번씩만 방문하고 다시 출발점으로 돌아오는 가장 짧은 경로를 '근사 알고리즘' 으로 찾아라
- 전제조건: 각 도시 사이의 거리는 삼각 부등식을 만족한다
- 도시 간 거리 데이터 (가중치):
 - $(A, B) = 10, (A, C) = 15, (A, D) = 20$
 - $(B, C) = 25, (B, D) = 25$
 - $(C, D) = 25$

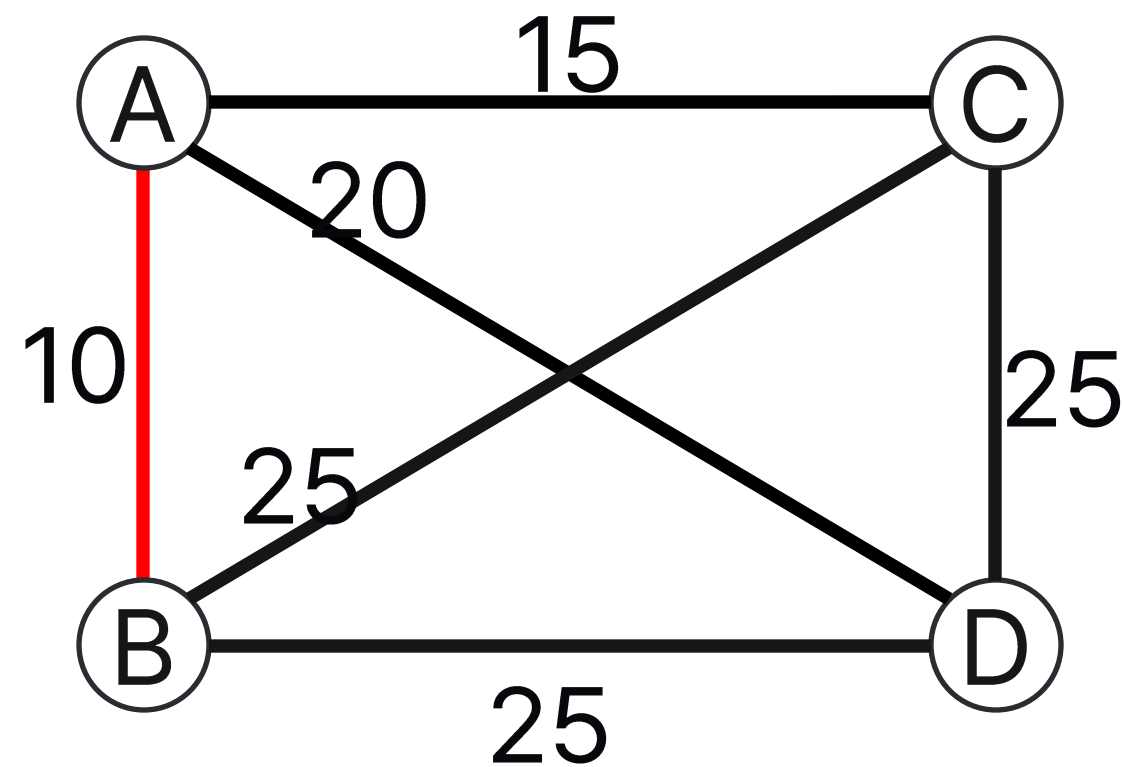
TSP 문제 예시

- 도시 간 거리 데이터 (가중치):
 - $(A, B) = 10$, $(A, C) = 15$, $(A, D) = 20$
 - $(B, C) = 25$, $(B, D) = 25$
 - $(C, D) = 25$



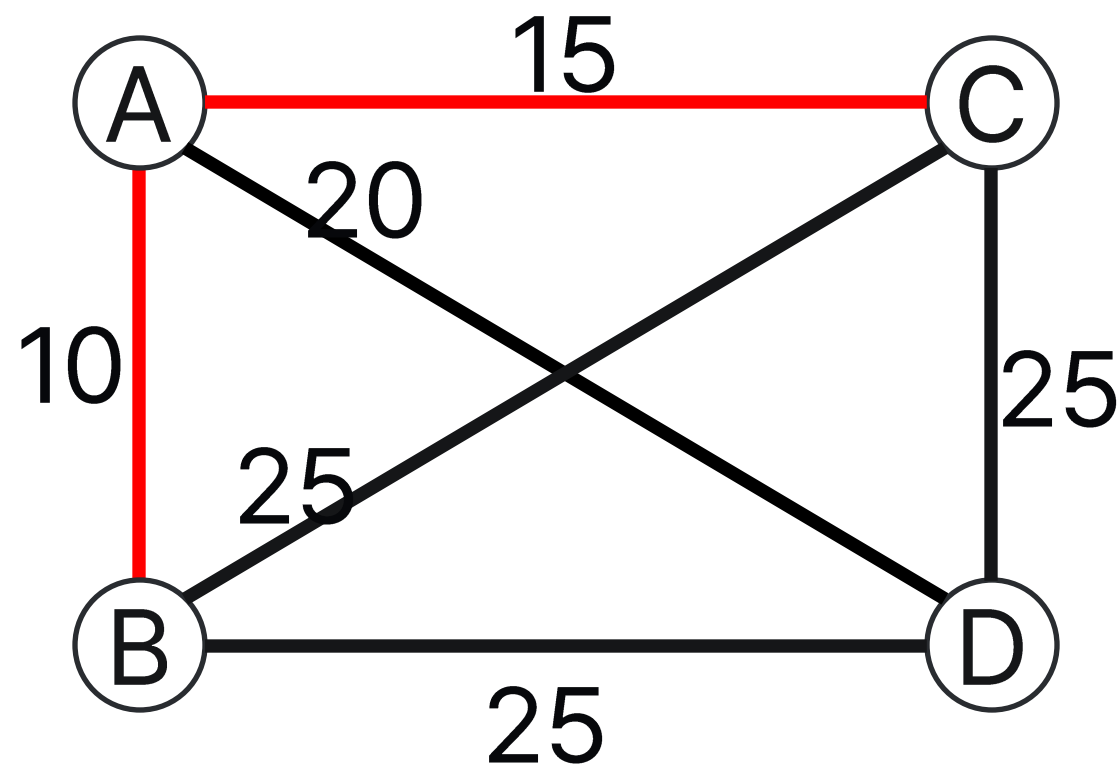
TSP 문제 예시

1. 주어진 그래프에서 MST 구축 (예시에서는 Kruskal 알고리즘 사용)
 - 가장 짧은 간선: (A,B)



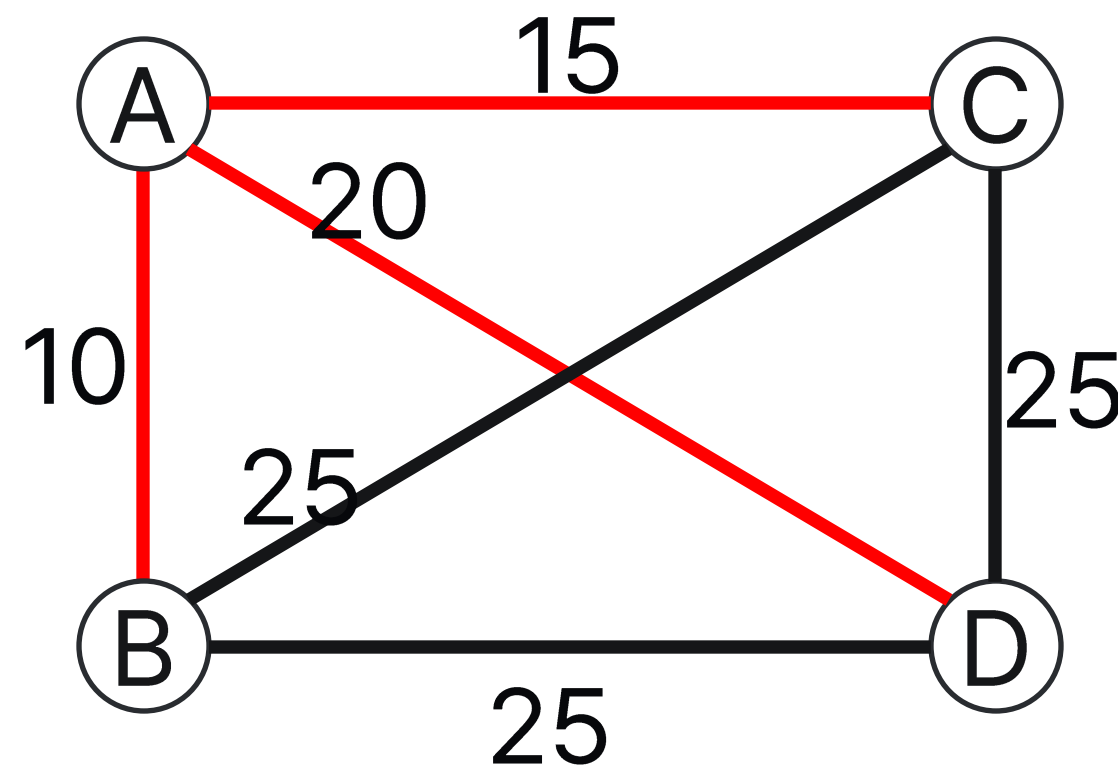
TSP 문제 예시

1. 주어진 그래프에서 MST 구축 (예시에서는 Kruskal 알고리즘 사용)
 - 가장 짧은 간선: (A,B) → 그 다음 짧은 간선: (A,C)



TSP 문제 예시

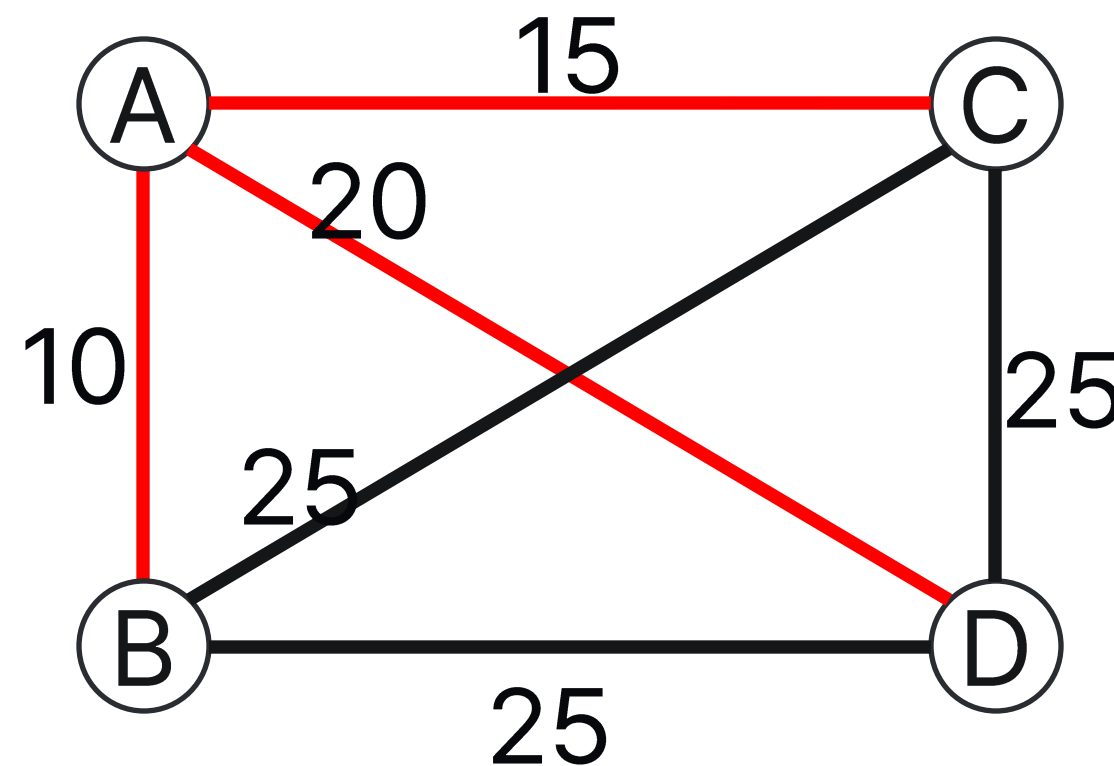
- 주어진 그래프에서 MST 구축 (예시에서는 Kruskal 알고리즘 사용)
 - 가장 짧은 간선: (A,B) → 그 다음 짧은 간선: (A,C) → (A, D) → 종료
 - MST 가중치 합: $10 + 15 + 20 = 45$



TSP 문제 예시

2. MST 간선을 '왕복' 한다 가정하고, 모든 노드를 방문하는 경로 생성

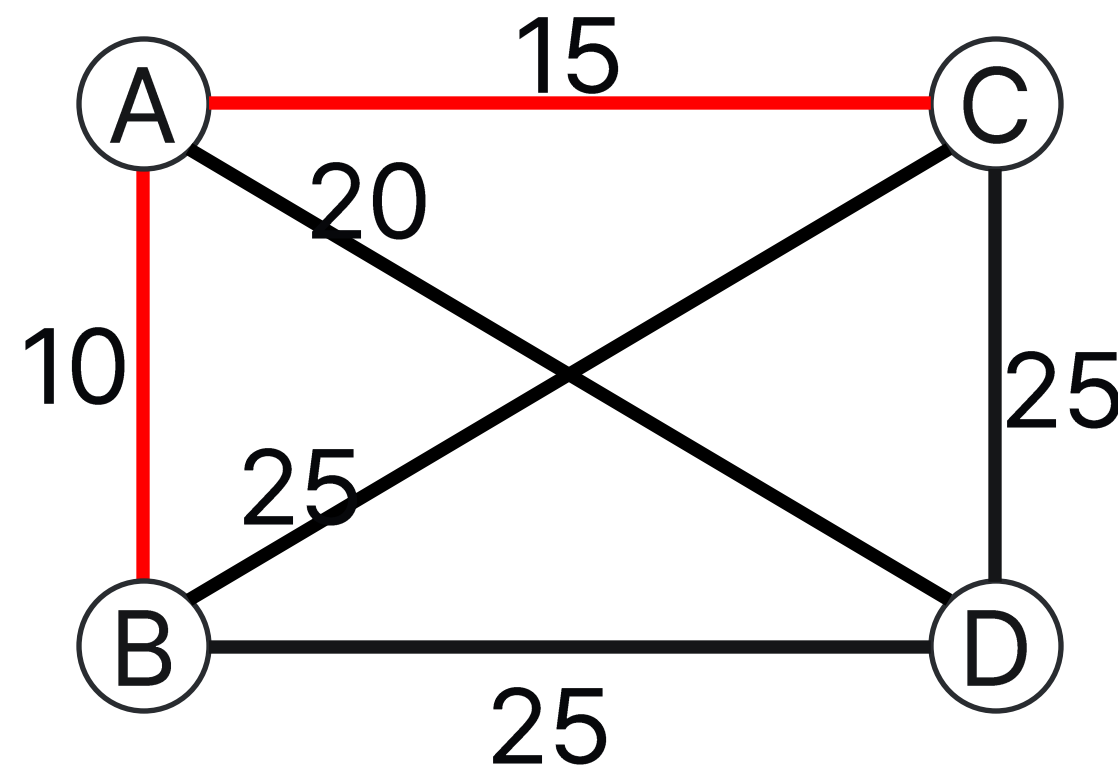
- 방문경로: $A \rightarrow B \rightarrow A \rightarrow C \rightarrow A \rightarrow D \rightarrow A$
- 이동경로 총 길이: $10 \times 2 + 15 \times 2 + 20 \times 2 = 90$ (2 x MST)



TSP 문제 예시

3. 지름길(Shortcut) 탐색: 중복 방문 도시는 건너뛰고, 방문하지 않은 다음 도시로 이동

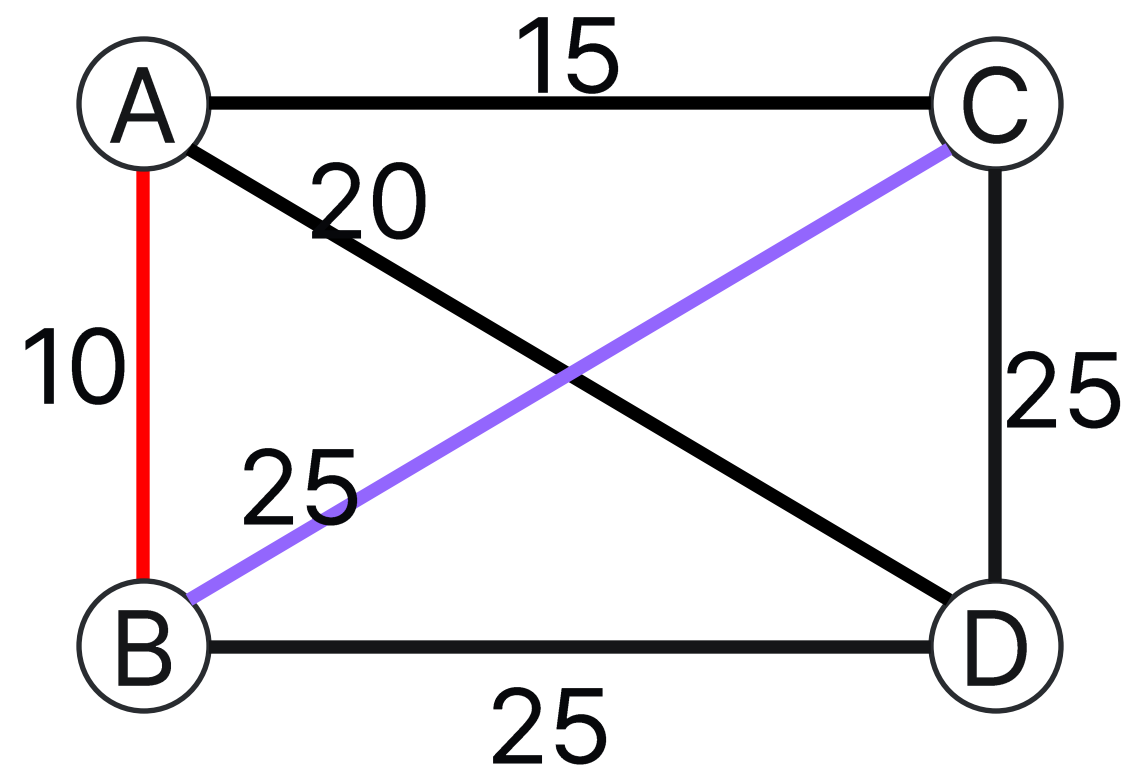
- 방문 경로: $A \rightarrow B \rightarrow A \rightarrow C \rightarrow A \rightarrow D \rightarrow A$
- 지름길 탐색: $B \rightarrow C$



TSP 문제 예시

3. 지름길(Shortcut) 탐색: 중복 방문 도시는 건너뛰고, 방문하지 않은 다음 도시로 이동

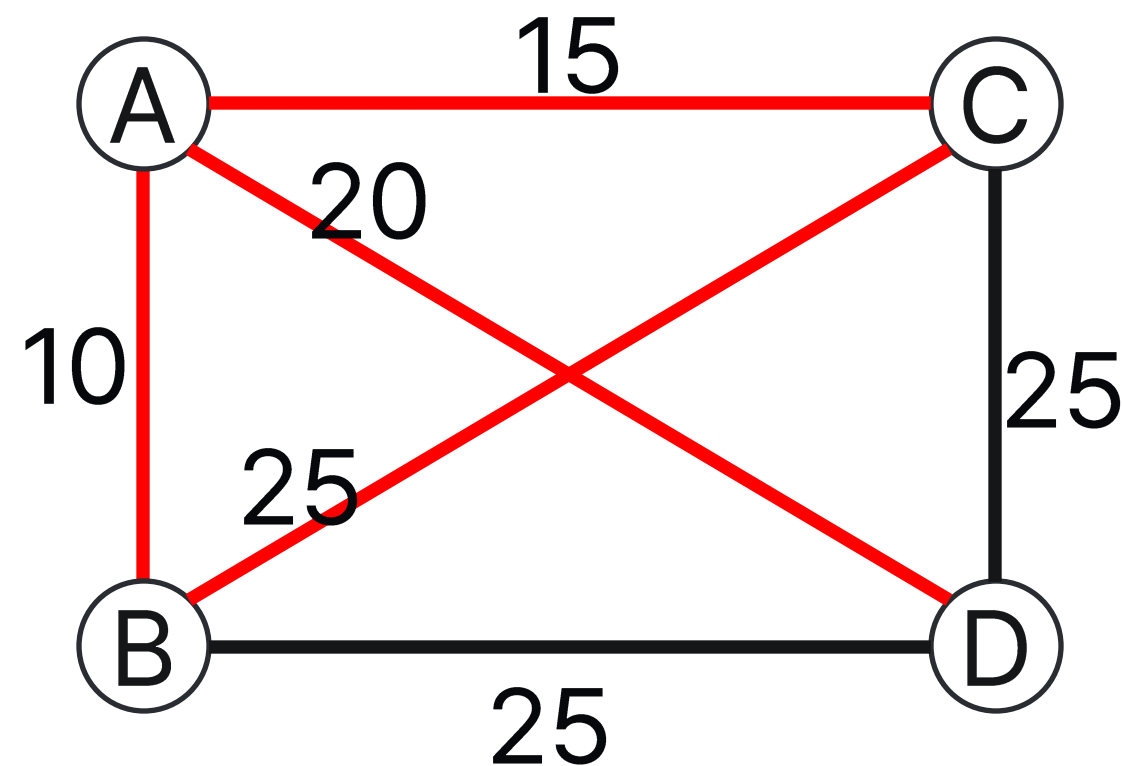
- 방문 경로: $A \rightarrow B \rightarrow C \rightarrow A \rightarrow D \rightarrow A$



TSP 문제 예시

3. 지름길(Shortcut) 탐색: 중복 방문 도시는 건너뛰고, 방문하지 않은 다음 도시로 이동

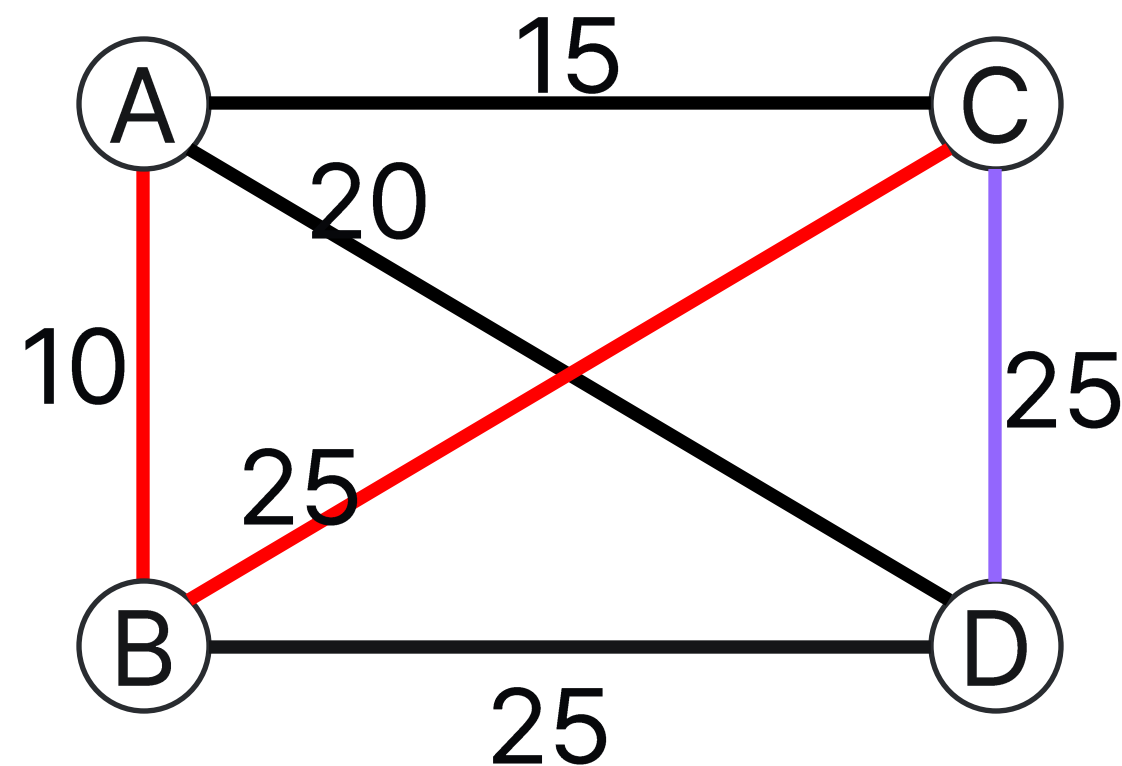
- 방문 경로: $A \rightarrow B \rightarrow C \rightarrow A \rightarrow D \rightarrow A$
- 지름길 탐색: $C \rightarrow D$



TSP 문제 예시

3. 지름길(Shortcut) 탐색: 중복 방문 도시는 건너뛰고, 방문하지 않은 다음 도시로 이동

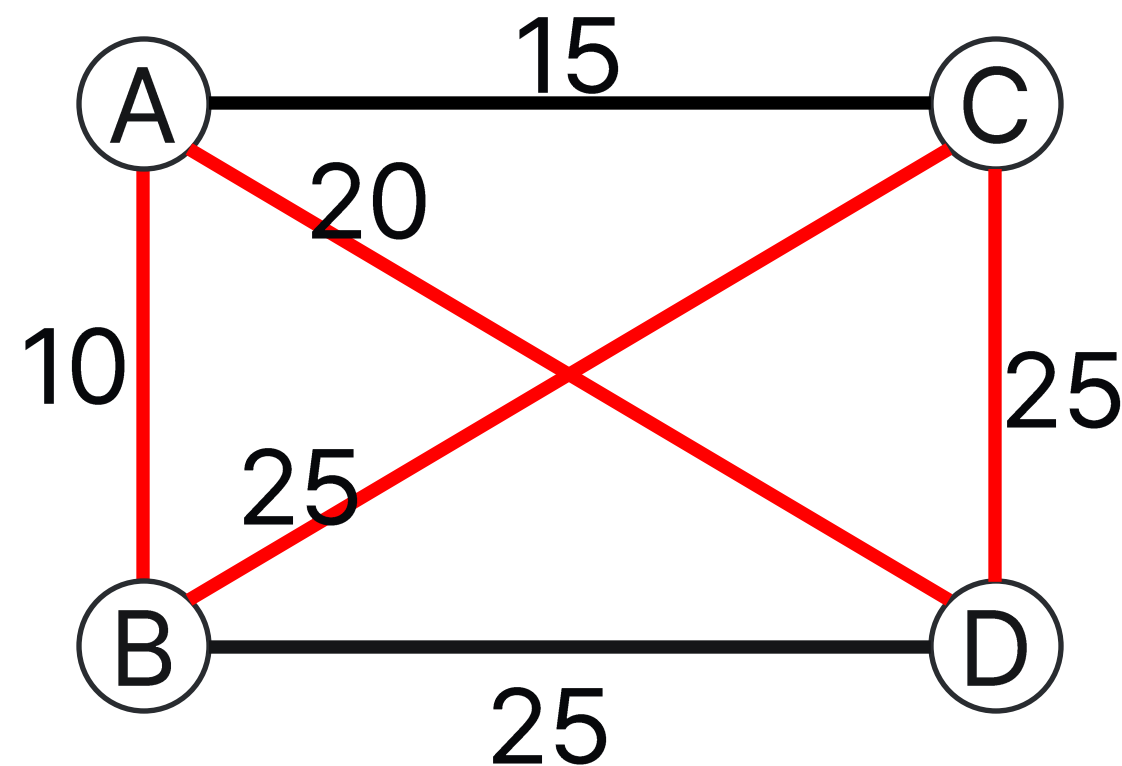
- 방문 경로: $A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$



TSP 문제 예시

3. 지름길(Shortcut) 탐색: 중복 방문 도시는 건너뛰고, 방문하지 않은 다음 도시로 이동

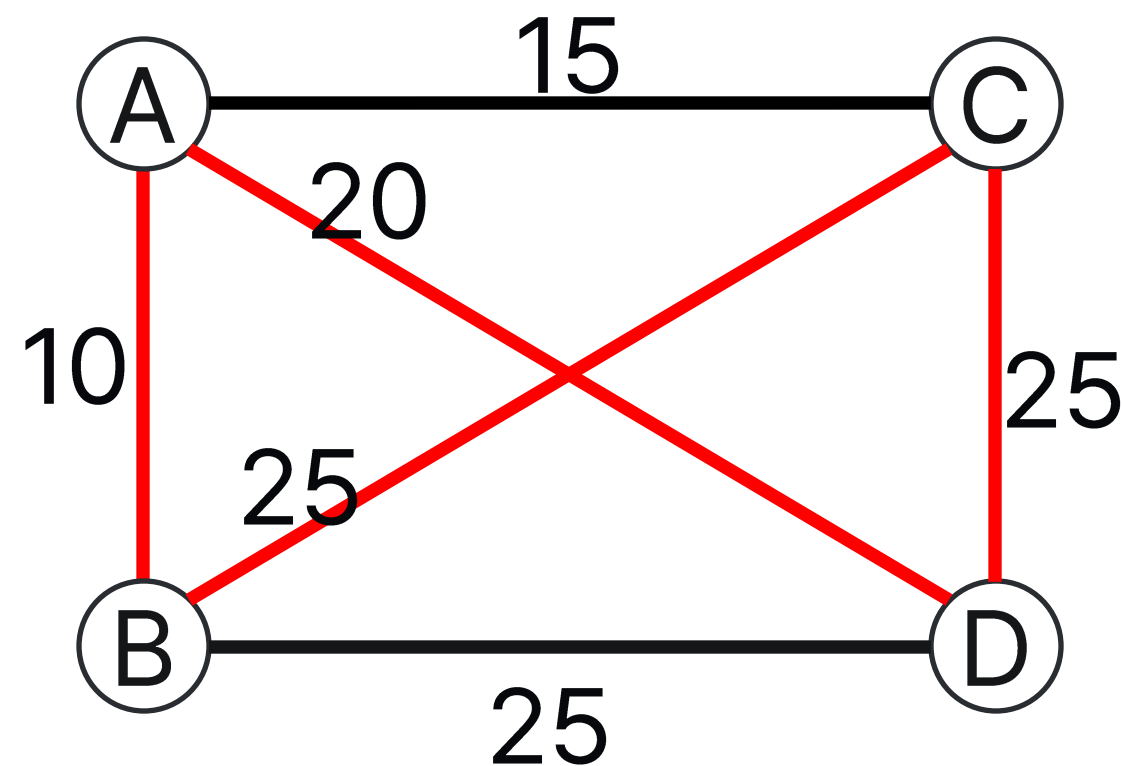
- 방문 경로: $A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$



TSP 문제 예시

4. 지름길을 적용하면, 중복 없는 TSP경로 완성

- 지름길을 적용한 최종 방문 경로: $A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$
- 근사해: $10 + 25 + 25 + 20 = 80$



근사비율 도출

- 증명1: MST는 최적해(C^*) 보다 작거나 같다
 - 최적해는 모든 정점을 연결하는 사이클, MST는 모든 정점을 연결하는 트리

$$MST \leq C^*$$

근사비율 도출

- 증명2: 2단계("MST 간선을 '왕복' 한다 가정하고, 모든 노드를 방문하는 경로 생성")의 경로 길이는 MST의 2배이다. 3단계(지름길)에서 구한 근사해(C)는 이 길이보다 작거나 같다.

$$C \leq 2 \times \text{MST}$$

근사비율 도출

- 증명1, 증명2의 결론을 종합하면, 아래와 같은 최종 결론에 도달

$$C \leq 2 \times C^*$$

→ 근사비율($p(n)$) = $C / C^* \leq 2$ (근사해는 최적해의 2배보다 길어지지 않는다)

대표적인 근사 사례: 0/1 Knapsack (배낭 문제)



0/1 Knapsack 문제 정의

- 배낭의 용량(W)이 제한되어 있을 때, 가치가 최대가 되도록 물건들을 담는 문제
- 단, 물건을 쪼갤 수 없음 (각각의 물건을 가방에 담거나(1), 빼거나(0) 만 할 수 있음)
- W 가 작은 경우, 물건의 개수(n)와 용량(W)에 비례하는 DP 해법($O(nW)$)이 있으나, W 가 큰 경우에는 불가능



0/1 Knapsack 문제 접근

첫 번째 접근방법: 단위 무게당 가치가 가장 높은 물건부터 담기 (Greedy 알고리즘)

- 물건을 쪼갤 수 없으므로 최적해를 보장하지 못하지만, “근사해”로서 적절해 보임
- 1단계: 모든 물건을 단위가치 기준으로 내림차순 정렬
- 2단계: 정렬된 순서대로 배낭에 담기
- 3단계: 다음 물건이 용량을 초과한다면, 남은 공간을 무시하고 종료



0/1 Knapsack 문제 접근

첫 번째 접근방법: 단위 무게당 가치가 가장 높은 물건부터 담기 (Greedy 알고리즘)

- 예시1
 - 배낭 용량 7, 물건 A(6kg, 13원), B(4kg, 8원), C(3kg, 6원)
 - 근사해: 13원 (물건 A)
 - 최적해: 14원 (물건 B, 물건 C)

➔ 근사 비율: 1.08



0/1 Knapsack 문제 접근

첫 번째 접근방법: 단위 무게당 가치가 가장 높은 물건부터 담기 (Greedy 알고리즘)

- 예시2
 - 배낭 용량 10, 물건 A(6kg, 7원), B(10kg, 10원)
 - 근사해: 7원 (물건 A)
 - 최적해: 10원 (물건 B)

➔ 근사 비율: 1.43 (오차가 매우 커질 수 있다!)



0/1 Knapsack 문제 접근

두 번째(개선된) 접근방법: Greedy 알고리즘과 단일가치가 가능 높은 물건의 가치 비교

- 첫 번째 Greedy 알고리즘의 약점을 극복하기 위해, 두 가지 후보 중 큰 값을 선택
 - 후보1. Greedy로 담은 물건들의 총 가치
 - 후보2. 단일가치가 가장 높은 물건의 가치



0/1 Knapsack 문제 풀이

1. 단위 무게당 가치가 가장 높은 물건부터 담기 (Greedy 알고리즘)
2. Greedy 알고리즘의 약점을 극복하기 위해, 두 가지 후보 중 큰 값을 선택
 - 후보1. Greedy로 담은 물건들의 총 가치
 - 후보2. 단일가치가 가장 높은 물건의 가치



0/1 Knapsack 문제 예시

- 문제: 아래와 같이 배낭의 용량(W)이 제한되어 있을 때, 가치가 최대가 되도록 물건들을 담아라
- 제약조건: 단, 물건을 쪼갤 수 없음 (물건을 가방에 담거나, 담지 않는 것만 할 수 있음)
- 배낭의 최대 수용 무게(W): 10kg
- 준비된 물건 리스트: 물건 A(6kg / 7만원), 물건 B(10kg / 10만원)



0/1 Knapsack 문제 예시

1. 단위 무게당 가치가 가장 높은 물건부터 담기 (Greedy 알고리즘)
 - 물건 리스트: 물건 A(6kg / 7만원), 물건 B(10kg / 10만원)
 - 단위 무게당 가치:
 - 물건 A: $7 / 6 = 1.2\text{만원/kg}$
 - 물건 B: $10 / 10 = 1\text{만원/kg}$
- ➔ 후보1의 근사해: 7만원(물건 A)



0/1 Knapsack 문제 예시

2. Greedy 알고리즘의 약점을 극복하기 위해, 두 가지 후보 중 큰 값을 선택

- 후보1. Greedy로 담은 물건들의 총 가치 → 7만원 (물건 A)
- **후보2. 단일가치가 가장 높은 물건의 가치 → 10만원 (물건 B)**

➔ 최종 근사해: 10만원(물건 B)



근사비율 도출

- 기본 전제: 최적해는 Fractional Knapsack(물건을 쪼갬)의 최적해 보다 작거나 같다
- 물건을 쪼갤 수 있다면, 배낭의 남은 공간을 알뜰하게 다 채울 수 있으니까!

$$C^* \leq C_{frac}$$



근사비율 도출

- 첫 번째 근사해(Greedy 알고리즘) 분석
 - 단위 무게당 가치가 큰 순서대로 물건을 담다가, 용량이 모자라 담지 못하게 되는 물건을 '물건 k' 라고 하면,
- 근사해(C_{greedy})에 '물건 k' 의 가치(v_k)를 더하면, C_{frac} 보다 크거나 같다

$$C^* \leq C_{frac} \leq C_{greedy} + v_k$$

근사비율 도출

- 첫 번째 근사해(Greedy 알고리즘) 분석
- 산술적으로 두 값(C_{greedy}, v_k) 중 적어도 하나는 최적해(C^*)의 절반보다 크거나 같다

$$C^* \leq C_{greedy} + v_k$$

$$\frac{C^*}{2} \leq \max(C_{greedy}, v_k)$$



근사비율 도출

- 최종 근사해(Greedy 알고리즘과 단일가치가 가장 높은 물건의 가치 비교) 분석
- 최종 근사해: $\max(C_{greedy}, v_k)$
- 첫 번째 근사해 분석결과: $\frac{C^*}{2} \leq \max(C_{greedy}, v_k)$

➔ 최종 근사비율($p(n)$) = $C^* / C \leq 2$ (근사해는 최적해의 최소 50% 이상이다)

근사 알고리즘의 장단점



장점

- 압도적인 계산 속도 (Efficiency):
 - NP문제를 다항 시간내에 푸는 것 이 가능
 - 입력 데이터가 많아져도 빠르게 문제를 풀 수 있음
- 구현의 단순함 (Simplicity):
 - Greedy와 같은 직관적인 전략을 주로 사용
- 수학적 품질 보장 (Theoretical Guarantee):
 - "최적해보다 $p(n)$ 배 이상 나쁘지 않다"는 근사 비율을 보장



단점

- 최적성 결여 (Lack of Optimality):
 - 운 좋게 최적해를 찾을 수도 있지만 대개의 경우 '충분히 좋은 해' 로 만족
 - 단 1%의 오차도 허용되지 않는 정밀 시스템(암호학, 물리 등)에는 부적합
- 증명의 난이도 (Difficulty of Proof):
 - 알고리즘 설계보다 '근사 비율'을 증명하는 과정이 어려움
 - 문제의 특성에 맞는 증명 기법이 필요하므로 학문적 진입 장벽이 존재
- 문제 의존성 (Problem Specific):
 - 모든 NP 문제에 통용되지 않으며, 각 문제마다 최적화된 고유의 근사 전략이 필요